

字符串算法

AC自动机

```
//dfs版
#include <cstdio>
#include <cstring>
#include <queue>
#include <vector>
using namespace std;

constexpr int N = 2e5 + 6;
constexpr int LEN = 2e6 + 6;
constexpr int SIZE = 2e5 + 6;

int n;

struct Node {
    int son[26];
    int ans;
    int fail;
    int idx;

    void init() {
        memset(son, 0, sizeof(son));
        ans = idx = 0;
    }
} tr[SIZE];

int tot;
int ans[N], pidx;

vector<int> g[SIZE]; // fail 树

void init() {
    tot = pidx = 0;
    tr[0].init();
}

void insert(char s[], int &idx) {
    int u = 0;
```

```

for (int i = 1; s[i]; i++) {
    int &son = tr[u].son[s[i] - 'a'];
    if (!son) son = ++tot, tr[son].init();
    u = son;
}
// 由于有可能出现相同的模式串，需要将相同的映射到同一个编号
if (!tr[u].idx) tr[u].idx = ++pid; // 第一次出现，新增编号
idx = tr[u].idx; // 这个模式串的编号对应这个结点的编号
}

void build() {
    queue<int> q;
    for (int i = 0; i < 26; i++)
        if (tr[0].son[i]) {
            q.push(tr[0].son[i]);
            g[0].push_back(tr[0].son[i]); // 不要忘记这里的 fail
        }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int i = 0; i < 26; i++) {
            if (tr[u].son[i]) {
                tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
                g[tr[tr[u].fail].son[i]].push_back(tr[u].son[i]); // 记录 fail 树
                q.push(tr[u].son[i]);
            } else
                tr[u].son[i] = tr[tr[u].fail].son[i];
        }
    }
}

void query(char t[]) {
    int u = 0;
    for (int i = 1; t[i]; i++) {
        u = tr[u].son[t[i] - 'a'];
        tr[u].ans++;
    }
}

void dfs(int u) {
    for (int v : g[u]) {
        dfs(v);
        tr[u].ans += tr[v].ans;
    }
}

```

```

    }
    ans[tr[u].idx] = tr[u].ans;
}

char s[LEN];
int idx[N];

int main() {
    init();
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        scanf("%s", s + 1);
        insert(s, idx[i]);
        ans[i] = 0;
    }
    build();
    scanf("%s", s + 1);
    query(s);
    dfs(0);
    for (int i = 1; i <= n; i++) {
        printf("%d\n", ans[idx[i]]);
    }
    return 0;
}

```

KMP算法

```
#include<bits/stdc++.h>
using namespace std;
void get_nxt(string s, int nxt[]){
    int j = 0;
    nxt[0] = 0;
    for (int i = 1; i < s.length(); i++)
    {
        while(j>0&& s[i]!=s[j]) j=nxt[j-1];
        if(s[i]==s[j])j++;
        nxt[i]=j;
    }
}
void solve(){
    int cnt=0;
    string s, t;
    cin >> s >> t;
    int* nxt = new int[t.length()];
    get_nxt(t, nxt);
    int j=0;
    for (int i = 0; i < s.length(); i++) //这里的s代表主串
    {
        while(j > 0 && s[i] != t[j]) j = nxt[j-1];
        if(s[i]==t[j]) j++;
        if(j==t.length()){
            cout<<i-t.length()+2<<endl;
        }
    }
    for (int i = 0; i < t.length(); i++){
        cout<<nxt[i]<<' ';
    }
}
int main(){
    solve();
}
```

Trie树

```
#include<bits/stdc++.h>
using namespace std;
int tree[3000005][70],pass[3000005],endp[3000005],cnt=1;
int getidx(char ch){
    // 映射函数
    if(ch>='a'&&ch<='z') return ch-'a';
    if(ch>='A'&&ch<='Z') return ch-'A'+26;
    return ch-'0'+52;
}
void insert(string s){//插入操作
    int cur=1;
    pass[cur]++;
    for(int i=0;i<s.length();i++){
        int idx=getidx(s[i]);
        if(tree[cur][idx]==0) {
            tree[cur][idx]=++cnt;
        }
        cur=tree[cur][idx];
        pass[cur]++;
    }
    endp[cur]++;
}
int search_whole_s(string s){
    //查询整个字符串出现次数
    int cur=1;
    for(int i=0;i<s.length();i++){
        int idx=getidx(s[i]);
        if(tree[cur][idx]==0) return 0;
        cur=tree[cur][idx];
    }
    return endp[cur];
}
int search_prefix_s(string s){
    //查询s作为前缀字符串出现次数
    int cur=1;
    for(int i=0;i<s.length();i++){
        int idx=getidx(s[i]);
        if(tree[cur][idx]==0) return 0;
        cur=tree[cur][idx];
    }
    return pass[cur];
}
```

```

}
void delete1(string s)
{
    if(search_whole_s(s)>0)
    {
        int cur=1;
        for(int i=0;i<s.length();i++)
        {
            int idx=getidx(s[i]);
            if(--pass[tree[cur][idx]]==0)
            {
                tree[cur][idx]==0;
                return;
            }
            cur=tree[cur][idx];
        }
        endp[cur]--;
    }
}
void clear()
{
    for(int i=1;i<=cnt;i++)
    {
        for(int j=0;j<=69;j++) tree[i][j]=0;
        pass[i]=0;
        endp[i]=0;
    }
    cnt=1;
}

```

string常用函数

三、基本属性

```
s.size();      // 长度（常用）
s.length();    // 同 size()
s.empty();     // 是否为空
```

四、访问与修改

```
s[i];          // 访问（不检查越界）
s.at(i);       // 访问（会检查越界）
```

```
s.front();     // 第一个字符
s.back();      // 最后一个字符
```

五、拼接与修改

1. 拼接

```
s += "abc";
s = s + "abc";
```

2. 插入

```
s.insert(2, "abc"); // 在下标2插入
```

3. 删除

```
s.erase(2, 3);     // 从下标2开始删除3个字符
```

4. 替换

```
s.replace(2, 3, "abc"); // 替换区间
```

六、子串操作

```
string t = s.substr(pos, len); // 从pos开始，长度len
```

示例：

```
string s = "abcdef";
s.substr(2, 3); // "cde"
```

七、查找操作（重点）

1. 查找子串

```
s.find("abc"); // 返回第一次出现的位置
```

找不到返回：

```
string::npos
```

示例：

```
if (s.find("abc") != string::npos) {
    // 找到了
}
```

2. 反向查找

```
s.rfind("abc");    // 从后往前找
```

3. 查找字符

```
s.find('a');
```

八、比较

```
if (s == t) {}
```

```
if (s < t) {}    // 字典序比较
```

底层是字典序 (lexicographical order)

九、排序

```
#include <algorithm>
```

```
sort(s.begin(), s.end());
```

示例:

```
string s = "dcba";
```

```
sort(s.begin(), s.end()); // "abcd"
```

十、反转

```
reverse(s.begin(), s.end());
```

十一、大小写转换

```
#include <cctype>
```

```
for (char &c : s) c = toupper(c);
```

```
for (char &c : s) c = tolower(c);
```

十二、统计字符

```
int cnt = count(s.begin(), s.end(), 'a');
```